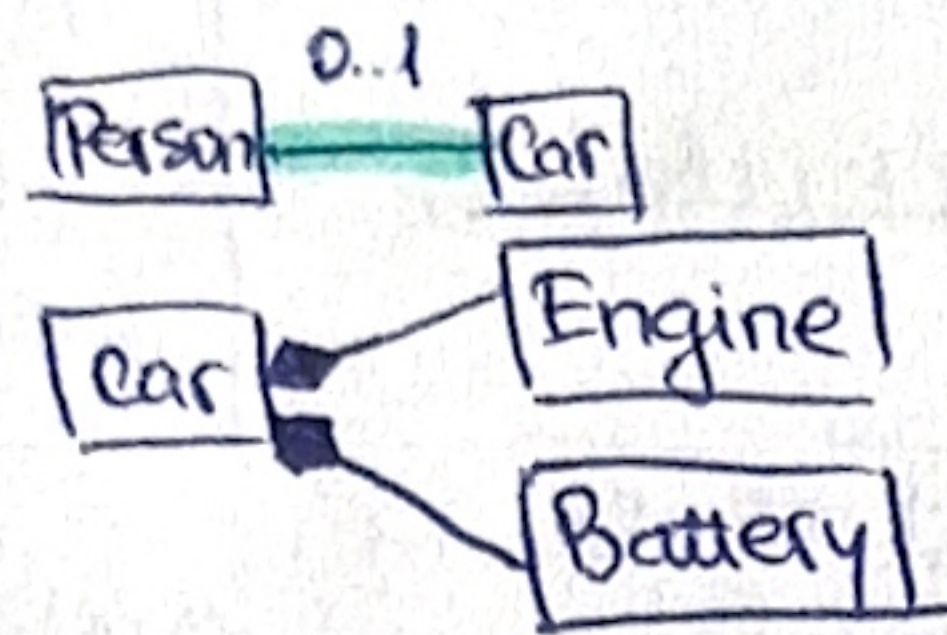
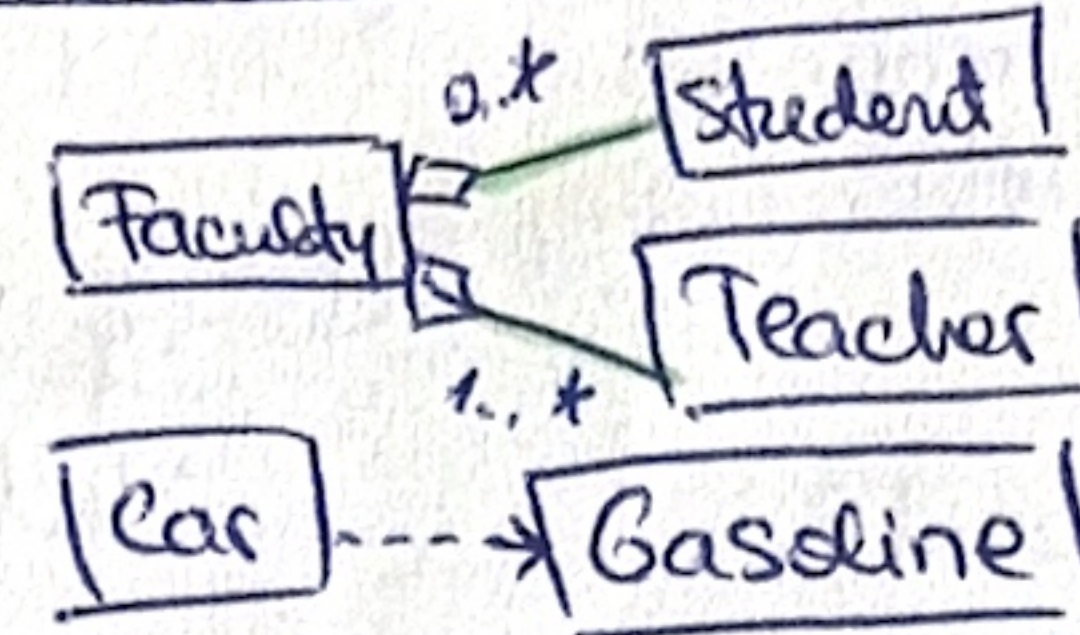


OOP - THEORETICAL REF. SHEET

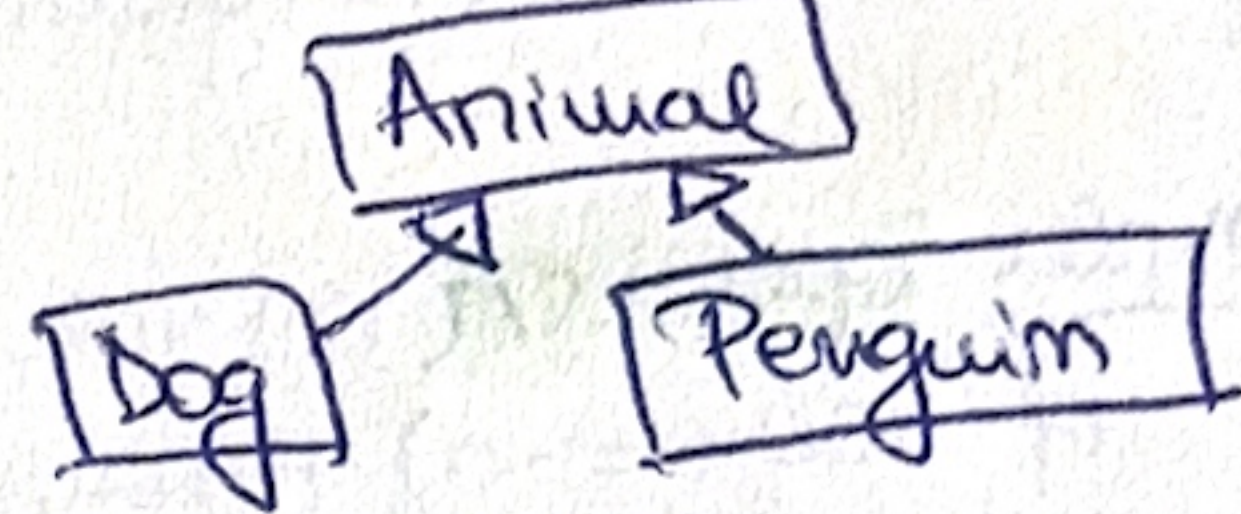
ASSOCIATION:
COMPOSITION:



AGGREGATION: (repo-coat)
DEPENDENCY:



INHERITANCE:



pointer/reference + virtual => REAL obj. type.
pointer/reference + non-virtual => POINTER TYPE.

- beverage => private member
0..* => zero or more (aggregation)
1..* => one or more (association)
0..1 => zero or one (association)

ex: B* b[] = { new B(), new D(), new D() };

catch-by-object
-> born at throw,
dies inside
catch after data
is copied

catch-by-value

born at
catch,
dies at
catch

dequeue = remove
+ return elem.
from the front

b[0] = B* => pointer type = B*
new B() => real obj. type = B
b[1] = B* => pointer type = B*
new D() => real obj. type = D

b[0] -> print()

NON-VIRTUAL

pointer type => B*
=> B::print()

b[1] -> print()

NON-VIRTUAL

pointer type => B*
=> B::print()

b[0] -> print()

VIRTUAL in base class.

REAL obj. type => B => B::print()

b[1] -> print()

Still VIRTUAL

REAL obj. type => D => D::print()

B b -> new object
B& b -> alias, no obj.
B* b -> alias, no obj.

bool operator == (const Complex& other) const
return this->real == other.real &&
this->imaginary == other.imaginary

Rule of three: -> if 1 one of these => when it goes out of scope => deletes twice => dangling pointer (if 1 copy constructor / assignment operator)

- 1) copy constructor
- 2) destructor
- 3) assignment operator

ITERATOR: itBegin = d.begin() => points to FIRST elem.
itEnd = d.end() => points AFTER the LAST elem.

v.insert(it, x) => inserts x BEFORE it.
*it = 11 => change VALUE.

double delete
(dangling ptr)

Shallow copy - copies the pointer
Deep copy - allocates new memory

-x => parameter
x = attribute
(distinction)

OBSERVATIONS:

- we can't create obj. from abstract classes!
- nothing after throw() gets executed!
(L => directly in catch (if finds cond.))
=> if not caught => error.
- if an object is thrown, it gets destroyed after catch.
- catch (Ex e) => catch BY VALUE => copy constructor.
- child class function f() is virtual as well if the parent class function f() is virtual (even if in the child class is not marked as virtual).
- d -> g(*b) => copy constructor.
- ! OUT OF SCOPE! => destructor automatically called!

By overriding a virtual fct. the child class completely replaces the base class behaviour.
If in base class the fct. prints "a" and it's virtual in child class, the overridden fct. prints "b" and it's virtual in child class.

str.erase(it);
str.insert(it, "b")

ERROR (uses invalid it.)

it = str.erase(it);
str.insert(it, "b")

CORRECT

end() - 1 = LAST ELEMENT

pop_back() -> delete LAST element.

v.erase(it);
it = v.begin() + 3;

CORRECT (iterators reset)

front_inserter(...) => inverse order.

back_inserter() => normal order.
When we have a vector of pointers, destructor is called automatically when main() goes out of scope; we need to do [delete d] only when we have sth. like: sumDiscount * d = new sumDiscount();
As long as the inheriting (child) class does not define pointers, it doesn't need to override the parent's destructor.

SHALLOW

Encoder -> Mixer

=> inside Mixer class: Encoder* e1
Encoder* e2

Sale -> SaleItem (0..*)
=> Sale contains 0 or several SaleItem (more)

=> inside Sale class: std::vector<SaleItem*> items;

SumDiscount -> Discount (1..*)
=> SumDiscount contains several Discounts

=> inside SumDiscount class: std::vector<Discount*> discounts;

MenuItem -> Action (0..1)

=> inside MenuItem class: Action* action; MenuItem(..., Action* action = nullptr);

Icecream -> IcecreamWithTopping

=> inside IcecreamWithTopping class: Icecream* icecream;

Expression

UnaryExpression

BinaryExpression

=> inside UnaryExpr: Expression* right; + inheritance from UnaryExpr.
=> inside BinaryExpr: Expression* left + inheritance from UnaryExpr.


```
bool operator==(const Complex& other) const {
    return this->real == other.real &&
           this->imaginary == other.imaginary;
}
```

```
Complex operator/(int value) const {
    if (value == 0) throw...
    return Complex { this->real/value, ... };
}
```

```
friend ostream& operator<<(ostream& out, const Complex& c) {
    out<< c.real << " + " << c.imaginary << "i";
    return out;
}
```

```
HTMLBuilder& operator+=(T elem) {
    if (...) throw...
    elems.push_back(elem);
    return *this;
}
```

```
Address& operator-
if (values.size() ==
    throw...
    values.pop_back();
    return *this;
}
```

```
Vector<T>& add(const T& elem) {
    elems.push_back(elem);
    return *this;
}
```

```
Address& operator+(T value) {
    values.push_back(value);
    return *this;
}
```

```
T& operator*(T value) const { return *this; }
```

Search dynamically:

```
ui->listWidget->clear();
std::string text = ui->lineEdit->text().toStdString();
if (text.empty())
    return;
auto question = service.get... (text);
ui->listWidget->addItem(QString...);
auto answers = service.get... (question);
for (const auto& a : answers) {
    ui->listWidget->addItem(QString::fromStdString(" " + a.toString()));
}
```

```
Question Service::getBestMatching(... text) {
    auto questions = repo.getQuestions();
    std::sort(q.begin(), q.end(), [this, text]
    (const Question& q1, ... q2) {
        return similarityScore(q1.getText(), text) >
               similarityScore(q2.getText(), text);
    });
    return questions[0];
}
```

← tab indentation.

```
connect(ui->lineEdit, &QLineEdit::textChanged, this, ...);
```

```
T sum() const {
    T s = values[0];
    for (int i = 1; i < values.size(); i++)
        s = s + values[i];
    return s;
}
```

```
template <typename T>
class Vector {
private:
    vector<T> elems;
public:
    ...
}
```

Clarify Handrail { ... } vs handrail { handrail { } etc.

I. child class needs all parents fields!

```
(H1)
class Parent {
private:
    int x; int y;
public:
    Parent(int x, int y) : x{x}, y{y} {}
};
class Child : public Parent {
public:
    Child(int x, int y) : Parent{x, y} {}
}
```

(H2)

```
class Child : public Parent {
private:
    int x, int y;
public:
    Child(int x, int y) :
        Parent{x, y} {}
};
NEVER!
Child(int x, int y) {
    this->x = x;
}
```

Copy constructor

II. parent fields private => provide getters/setters; protected => no getters/setters.

III. Child passes extra fields on its constructor: Child(int x, int y, int extra): Parent{x, y, extra} {}

IV. Dependency injection: class PinkBag: public Bag { private: GoldChain* chain; public: PinkBag(..., GoldChain* chain): ..., chain{chain} {} }

V. Internal composition in class Beverage (...): descrip{descrip} {} Beverage with Milk(Beverage* beverage, int cnt): Beverage{"w milk", beverage{beverage}, cnt{cnt}} = nullptr => optional param -> at the END (last field of the constructor!)

template <typename T>

class ... {
private:

std::vector<T> elems;

Patterns

when you see `for (auto x: todo) → you need function for begin() and end()!` `auto begin() { return elems.begin(); }`

`auto begin() { return elems.begin(); }`

`auto end() { return elems.end(); }`

operator ++

Adder& operator++()

values.push_back(values.back());
return *this;

}

friend ostream& operator<<(ostream& out, const Adder& a)
{ out<<"Activity"<<a.name<<" at "<<a.hour;
return out;
}

Stack<T> operator+(const T& elem) const {

if ... throw ...
Stack<T> result = *this;
result.elems.push_back(elem);
return result;

template <typename ID,

typename T>

class Repo {

private:

std::vector<std::pair<
ID, T>> elems

operator ==

bool operator==(const Complex& other) const {

return this->real == other.real &&
this->imag == other.imag;

bool operator==(const SmartPointer<T>& other) {
return *ptr == *other.ptr;

MyObjects& addBeginning(Object* object) {
objects.insert(objects.begin(), object);
return *this;

operator +=

HTMLBuilder& operator+=(T elem) {

if (elem == nullptr) throw ...

elems.push_back(elem);
return *this;

ToDo<T>& operator+=(const T& elem) {

elems.push_back(elem);
return *this;

operator =

SmartP<T>& operator=(const SmartP<T>& other)

{ if (this == &other) return *this;

(*count)--;

if (*count == 0) { delete ptr; delete count; }

ptr = other.ptr;

count = other.count;

(*count)++;

return *this;

void add(ID id, const

T& obj) {

elems.push_back({id, obj});

max

T fct(const vector<T>& v)

{ T max = v[0];

for (const T& elem: v) {

if (elem > max)

max = elem;

return max;

}

T pop() { (stack)

T e = elems.back();

elems.pop_back();

return e;

T pop() { (queue)

T e = elems.front();

elems.erase(elems.

begin());

return e;

operator /

Complex operator/(int value) const {

if (value == 0) throw ...

return Complex { this->real/value,
this->imag/value };

operator <<

inside class: ↓

friend ostream& operator<<(ostream& out,
const Complex& c)

outside class: ↓

ostream& operator<<(ostream& out,
const Complex& c) {

out<<c.real<<" " <<c.imag<<"i";

return out;

reverse print

for (auto it = elems.rbegin();
it != elems.rend(); ++it) { out<<*it<<"\n";

operator +

Adder& operator+(T value) {

values.push_back(value);
return *this;

}

Set<T> operator+(const T& elem) const {

Set<T> result = *this;

for ... throw ...

result.elems.push_back(elem);

return result;

}

operator *

T& operator*() const {

return *ptr;

}

char operator*() const {

return value;

operator -

Vector<T> operator-(const T& elem) {

Vector<T> result;

bool found = false;

for (...) {

if (!found && current == elem)

found = true;

else result.add(current);

}

operator --()

Adder& operator--() {

if (values.size() == 1)

throw ...

values.pop_back();

return *this;

}

bool operator+=(

const T& e) {

elems.push_back(e);

return *this;

operator []

T operator[](int i) {

return elems[i];

}